

CANDY

DIALOGUE ENGINE

Input Commands

1.1.0

During dialogue, you'll probably want to let the player provide input, such as entering their name, typing a code, or so on. This can be done with the Input commands.

§Mouse

The §Mouse command changes the MOUSE_MODE in Godot. The standard values are available:

Visible -- The cursor is visible and can exit the window.

Hidden -- The cursor is invisible and can exit the window.

Captured -- The cursor is hidden and can't move, but mouse moves objects (e.g. FPS camera).

Confined -- The cursor is visible but can't exit the window.

Confined_Hidden -- The cursor is invisible and can't exit the window, but it can move.

These modes are built into Godot, so refer to the Godot documentation for more details.

§Player_Advance

The §Player_Advance command pauses dialogue until the Player presses the action assigned to input_advance_dialogue (default: Dialogue_Advance).

This is similar to how players can press the same key to advance dialogue after a character has spoken. However, this command enables dialogue to require manual advancing after any line.

Auto-Advance settings have no effect on this command: those settings are only for Spoken Lines.

Manual advancing will not work if input_enabled is set to 'false' in the engine settings, or if no action is assigned to input_advance_dialogue. **This can cause dialogue to get stuck!**

The command does not feature safety features to guard against this risk: this is because in some special cases, users may want to set input_enabled to 'true' or assign an action to input_advance_dialogue later at runtime, when the command is already running.

§Input

The §Input command displays an input window of your choosing, where the player can enter data in any way you decide: typing, key or button presses, mouse movement, drag-and-drop...

How you use the §Input command is up to you: you can display a prompt to let the player enter their name, a number pad to type a code, some kind of puzzle where they must place objects in the correct slots...

Overall, think of the §Input command as a tool to have the player *do something* during dialogue, and perhaps let the player's actions influence how the dialogue proceeds.

Configuration

The thing about the §Input command is that it doesn't actually do much. Candy DE provides a default input window as a template, but it's really up to you to create the input windows you need.

Which is a good thing: it means you're not limited in the types of inputs you can use. If you want the input to be a puzzle to solve, you can do that. If you want it to be a 2-way radio interface, with dials for changing frequency, you can do that. If you want a hexadecimal keypad, you can do that. If you want a CCTV camera controller, you can do that. If you want a sound mixing table...

Yes, that's A LOT of possibilities, and there are many more. Not to mention the thousands of ways each could be implemented. Candy DE would never be able to anticipate and provide for every project. So instead of locking you into a few pre-built input menus, it makes you build your own.

So what does the §Input command actually do?

It loads and opens an input window/menu (or if it's already loaded, it displays it), and it calls a function inside of it to configure it.

To configure the menu, it passes the command's data to it:

- **input_variable** -- the variable where player input should be stored when finished.
- **setup_mode** -- the method to use to configure the menu.
- **input_instructions** -- custom instructions to display on screen for the player.
- **input_placeholder** -- potential placeholder text inside an input field.
- **input_default_text** -- optional pre-written input.

These variables aren't always relevant: it depends on the kind of input UI being used. Candy DE doesn't care what these variables or whether you even provide values for them: it just passes them to the input UI so it can configure itself.

Default Menus

The Input Menu provided with Candy DE, is a simple text input UI, where the player can type text with the keyboard.

To use it, just set the following values in the `$Input` command:

File: "Default" (without quotation marks). To fetch the data from a variable at runtime, provide a variable reference with the normal variable symbols (default '£', '\$' or '€').

Mode: "Default" (without quotation marks). To fetch the data from a variable at runtime, provide a variable reference with the **super** symbols (default '££', '\$\$' or '€€').

Variable: Write the variable where the input should be saved, prefixed with the appropriate variable symbols. For example, to save the input to 'global.player_name', write '£global.player_name'. To fetch the data from a variable at runtime, provide a variable reference with the **super** symbols (default '££', '\$\$' or '€€').

Instructions: Write what kind of input the player should provide (e.g. "Enter your name."). To fetch the data from a variable at runtime, provide a variable reference with the **super** symbols (default '££', '\$\$' or '€€').

Placeholder: Write optional placeholder text (e.g. "Name"). To fetch the data from a variable at runtime, provide a variable reference with the **super** symbols (default '££', '\$\$' or '€€').

Text: Write a default suggested input (e.g. "John"). To fetch the data from a variable at runtime, provide a variable reference with the **super** symbols (default '££', '\$\$' or '€€').

Custom Menus

Candy DE has barely any requirements from input menus: they're elements of your game, not Candy DE. All Candy DE does is:

- Display a menu when prompted,
- Pause dialogue,
- Wait for a signal to resume.

Even the `setup()` function is optional: it's only purpose is to inform the menu that it's being 'activated' (i.e. used by the player), in case your menu design requires it.

In other words, Candy DE is completely hands-off when it comes to input menus.

If you're new to coding or Godot and you're concerned about having to design input menus yourself: normally, if you can make the game you're trying to make, you can make input menus. You just need to ask yourself what your game is meant to do with the player input you're designing a menu for.

- Player name? Change the player name in your game variables!
- Code to a door? If the code is correct, toggle the door as unlocked in your game variables!
- Slot machine? Decrease player gold when they pull the lever, increase it if they win!

- And once the input menu is closed: emit the `caller.input_received(depth, "finish", null)` signal, with those exact arguments, so Candy DE knows to continue the dialogue!

That's all it is. Input menus are just another game mechanic, not a part of Candy DE or dialogues.

That being said, when creating your own input menus, you can assign them the `Input.gd` script provided with Candy_DE: this script is designed to be expandable with any behavior that any input menu may require, thanks to match statements inside each important function.

Setup()

In the `Input.gd` script, the `setup()` function is called when Candy DE processes an `$Input` command, and it determines what happens when the input menu is displayed to the player. This function is not required: if your input menu script doesn't include it, Candy DE won't call it and won't crash.

The function contains a match statement, where you can freely add your own configuration setup modes. You can then tell an `$Input` command which one to use by entering it in the "Mode" field.

Whatever setup modes you add to the function, they should probably do the following:

- a) Save which variable the input should be saved to upon submission (`save_variable`).
- b) Tell the `config()` function what configuration mode to use (`config_mode` variable).
- c) Tell the `submit()` function what special behavior to perform (`submit_mode` variable).
- d) Tell the `cancel()` function what special behavior to perform (`cancel_mode` variable).
- e) Display the instructions, placeholder or pre-written text inside the input menu.
- f) Hide or show the cancel and confirm buttons.
- g) Set the `MOUSE_MODE` to what is appropriate.
- h) Call the `config()` function to continue.

The "Default" setup mode already does all this, so you can use it as a template, or just re-use it outright if you don't need to make any changes to it for your own purposes.

To be clear, every input menu doesn't need its own setup mode: if a mode works for multiple input menus, they can all use the same one.

Config()

The `config()` function configures secondary behaviors of the input menu.

The "Default" mode is designed for a text input: it configures the character and word limits, the blacklisted or whitelisted words, a potential regex, allowed cases and characters, whether error

messages should be displayed if an input is invalid, and whether the enter key submits the input or adds a line break.

To be precise, the "Default" configuration mode doesn't impose any restrictions: it allows everything by default. If you want to restrict written input, you can copy it, give it another name, and configure the restrictions you want. For example, when the player is prompted to enter their name, you might want to disallow numbers or special characters.

You can also add your own restrictions and settings in the form of variables. Depending on what your input menu does, you'll have to write your own function to handle custom settings, or expand the `check_input()` function.

Check_Input()

The `check_input` function verifies that the input provided conforms to any restrictions and requirements you've chosen.

By default, it checks the restrictions and requirements mentioned above in the `Config()` section.

If you want to create your own rules, just add code for them in the `check_input()` function.

By default, it's called automatically by `_on_input_changed()`. Or in other words: anytime the player changes the text input. If your input menu requires checking that an input is valid, make sure to call `check_input()` as needed.

Submit()

The `submit()` function should be called when input is submitted: either when the player presses a key or clicks a button, or perhaps when a timer expires.

By default, if `check_input()` doesn't flag the input as invalid, `submit()` just calls `store_to_variable()` to save the input in the appropriate variable, and it emits the `caller.input_received(depth, "finished", null)` signal to let Candy DE know it can resume.

However, if you require special behavior, `submit()` features a match statement. Remember to setup `submit_mode` in `setup()`, or possibly in `config()` if that makes more sense to your case.

`submit()` can also close the input menu in two ways: by default it hides it. However, you can define special situations in the match statement that will set `delete_menu` to 'true'. If so, the menu will be deleted instead.

Cancel()

The `cancel()` function is called when the input is canceled. By default, it hides the input menu and emits the `input_finished()` signal so Candy DE can resume dialogue.

However, `cancel()` also features a match statement if you want to account for special circumstances, or program special behaviors.

It also features a variable, `cancel_valid`, which is 'true' by default. If 'false', it won't allow cancellation: the menu will remain open and dialogue will not resume. Use the match statement to set it to 'false' in situations that may require it.

Like `submit()`, `cancel()` can also close the input menu in two ways: hiding it or deleting it. Set `delete_menu` to 'true' through the match statement to delete it.

Store_Variable()

The `store_variable` function simply stores the input into the variable indicated in `var save_variable`.

The value of `save_variable` is a string, so `store_variable()` interprets it so it can save the input in the right place.

Saving input to a variable is optional: you can save it and let some other function do something with it, or you can let the input menu script directly use or apply the input was appropriate. It all depends on the type of input and the purpose of your input menu.

Naming Requirements

Very important: the `$Input` command displays the input menu if it's already loaded in the `Candy_UI` scene, or loads it from resource files if it isn't.

For this reason, the root node of your input menus must have the exact same name as the `.tscn` scene file: if the node is named "Keypad", the scene must be saved as 'Keypad.tscn'.

Input menus must also be stored inside the correct folder. By default:

"Candy_DE/Scenes/Input Menus"

If you want to save input menus in a custom folder, make sure to update the path in `Candy_Variables.gd`, in the variable named 'input_menus_folder'.

Lastly, if you preload input menus in your game, they must be children of the "Input" node, inside the `Candy_UI` scene. The full path would be:

`$"Candy_UI/Input/Your_Menu"`

Remember that the `Candy_UI` scene is an autoload, accessible with the prefix 'candy_ui'.

Signal: input_received()

This signal is used to let input menus communicate with the dialogue engine. In the default input menu provided, it is used to inform the engine that we're finished with the input menu and dialogue can resume:

`caller.input_received(depth, "finish", null)`

However, you can use the signal to trigger other behaviors in the dialogue engine. To do this, open `candy_engine.gd` and navigate to the Commands section:

There, look for the §input command:

Scroll down to Step 7, and look at the 'match event_type' statement:

Notice that "finish" is present there. Add any other values you need, then return to your input menu script.

In your input menu script, whenever you want to trigger custom behavior in the dialogue engine, emit the signal:

```
caller.input_received(depth, "event", null)
```

Replace "event" by whatever value you want (i.e. one of the values in the 'match event_type' statement from earlier), and optionally replace 'null' with any variable you want to act as a parameter (use an array if you need to pass multiple values).

What can you use this for? Only you can know, this is dependent on your project. But as an example, if you look at the §Choice_List command, it has a similar 'while' loop, where it awaits the choice_list_event() signal, as well as a similar 'match' statement. The §Choice_List command uses that match statement to trigger various dialogue commands, and you can copy that logic into the input command's 'match input_event' statement (it may require some adaptation).